Subquadratic Sparse Attention: Content-Routed Exact Attention for Long Contexts

Jon Smirl

July 2, 2026

Abstract

Dense self-attention costs $O(n^2)$ in the sequence length n, which is the binding constraint on long-context language models. This paper presents *Subquadratic Sparse Attention* (SSA), an attention mechanism that, for each query, (i) routes to a small content-dependent set of key blocks using only per-block summary statistics, (ii) adds a local window, and (iii) performs *exact* softmax attention over the selected keys. The per-query work is $O(\kappa)$ in a fixed budget $\kappa \ll n$ plus a sublinear routing cost, so the layer runs in $O(n\sqrt{n})$ flat or near-linear with a hierarchical router. A supporting theory establishes when this is sound. A retrieval-margin analysis shows that softmax attention recovers a target key with weight $\sigma(\beta\Delta - \log\mu)$, where Δ is the score margin and μ the number of competing distractors; selection works because it cuts μ from n to κ, making recovery *flat in* n. The analysis gives the admissible routing bound that licenses summary-only selection, a tempered (cumulant) routing score that—unlike centroid routing—sees in-block outliers, and a closed-form variance prune test from Samuelson's inequality. A limit is then proved: cheap and lossless selection cannot hold for arbitrary keys (a one-line probe argument); adding length-robustness gives the trilemma (at most two of the three). So SSA's subquadratic *exactness* is licensed by the *benign geometry* of trained representations—geometry that training can be made to manufacture, via a routability regularizer that shrinks off-target spread. Finally, length generalization (rotary position plus staged continued training reaches 32× the trained length at 0.98 recall for ∼800 adaptation steps) and a construction pipeline that converts a dense pretrained model into a subquadratic one by swapping the attention and briefly adapting (recovering to within +1.2 perplexity of a dense model given equal training while attending 38% of keys) are demonstrated. All claims are accompanied by measurements at controlled scale.

1 Introduction

A transformer layer computes, for queries $Q \in \mathbb{R}^{n\times d}$, keys $K \in \mathbb{R}^{n\times d}$, and values $V \in \mathbb{R}^{n\times d}$,

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\Big(\tfrac{QK^\top}{\sqrt{d}}\Big)V.$$

The $n \times n$ score matrix makes both time and memory $\Theta(n^2 d)$. For contexts of 10^6–10^7 tokens this is prohibitive, yet most of the matrix is near-zero: for a given query only a small set of keys carries appreciable weight. The question is whether one can *find* that set without forming all n^2 scores.

Two subquadratic families answer differently. *Kernel / linear attention* replaces $\exp(\langle q, k\rangle)$ by a factorizable feature map $\phi(q)^\top \phi(k)$, giving $O(n)$ cost but a low-rank (smoothed) attention matrix. *Sparse / selective attention* keeps the exact softmax but evaluates it only on a chosen subset of keys. SSA is in the second family, with three design commitments:

1. **Content-dependent selection.** The chosen keys depend on the query, not only on position; this is what lets a query reach the one relevant block a million tokens back.

2. **Summary-only routing.** Selection uses per-block statistics (a mean, a covariance), so it does not read every key—that is where the subquadratic saving comes from.

3. **Exact attention over the selected set.** Within the selected keys the softmax is computed exactly, so there is no kernel-approximation error on the keys that matter.

The paper is organized around one tension. Selection is cheap only if it can judge a block from its summary; it is *correct* only if those summaries do not hide a key that mattered. Sections 3–5 make both sides precise; Section 6 shows they cannot both hold in the worst case, so something must give; Section 7 shows what gives—the data geometry, which training shapes. Sections 8–9 turn the mechanism into a usable recipe.

2 Notation and the retrieval view

Fix a single query $q \in \mathbb{R}^d$ and keys $k_1, \ldots, k_n$. Write the *logit* (score) of key j as

$$a_j \;=\; \langle q, k_j\rangle, \qquad w_j \;=\; \frac{e^{\beta a_j}}{\sum_{l=1}^{n} e^{\beta a_l}}, \qquad \beta = \tfrac{1}{\sqrt{d}},$$

so w is the attention distribution and β its inverse temperature. The output is $o = \sum_j w_j v_j$.

A long-context layer is, operationally, an *associative recall*: a query must place most of its weight on the key(s) that hold the information it needs and little on the rest. Attention is therefore analyzed by the weight it puts on a designated *target* key $k_\star$ relative to the *distractors* $\{k_j\}_{j \neq \star}$.

3 Retrieval margin and the recovery weight

Define the *margin* of the target as the gap between its logit and the typical distractor logit,

$$\Delta \;=\; a_\star - \bar{a}_{\text{dist}}, \qquad \bar{a}_{\text{dist}} = (\text{typical } a_j,\ j \neq \star).$$

Suppose there are μ effective distractors with logit $\approx a_\star - \Delta$. Then the target weight is

$$w_\star \;=\; \frac{e^{\beta a_\star}}{e^{\beta a_\star} + \mu\, e^{\beta(a_\star - \Delta)}} \;=\; \frac{1}{1 + \mu\, e^{-\beta\Delta}} \;=\; \sigma\big(\beta\Delta - \log\mu\big), \tag{3.1}$$

where $\sigma(x) = 1/(1+e^{-x})$ is the logistic. Equation (3.1) is the *recovery weight*. Two consequences:

- **Detectability threshold.** The target is recovered ($w_\star > \frac{1}{2}$) iff

$$\boxed{\beta\,\Delta \;>\; \log\mu} \tag{3.2}$$

  Recovery degrades only *logarithmically* in the number of distractors.

- **Why selection helps, and why it is flat in n.** Dense attention pays $\mu = n$. A selector that attends only a budget of κ keys pays $\mu = \kappa$, replacing the threshold $\log n$ by $\log\kappa$. If κ is held *fixed* as the context grows, the threshold (3.2) does not move with n: retrieval accuracy becomes *independent of context length*. This is the entire value proposition of selection, and it is why a fixed-budget selector can answer single-target queries at 10^6–10^7 tokens that dense attention, with its $\log n$ erosion, increasingly struggles with.

The catch is hidden in the phrase "a selector that attends κ keys": the selector must *contain the target in its budget*. Sections 4–6 are about exactly that.

4 The algorithm

4.1 Block partition and summaries

Partition the n keys into B contiguous *blocks* of size b (so $Bb = n$). For block c precompute, once, the summary statistics

$$\mu_c = \frac{1}{b}\sum_{j\in c} k_j \quad \text{(mean)}, \qquad \Sigma_c = \frac{1}{b}\sum_{j\in c}(k_j - \mu_c)(k_j - \mu_c)^\top \quad \text{(covariance)}, \tag{4.1}$$

and a radius $R_c = \max_{j\in c}\|k_j - \mu_c\|$. In practice Σ_c is kept diagonal, $\sigma_c^2 = \operatorname{diag}\Sigma_c$, so a block's summary is $O(d)$ numbers.

4.2 Routing score

For a query q, the *routing score* of block c estimates the largest logit the block can contribute. The exact quantity is the block's log-sum-exp $\mathrm{LSE}_c(q) = \log\sum_{j\in c} e^{\beta\langle q, k_j\rangle}$, which requires reading every key. SSA uses its *second-order (cumulant) surrogate*, computable from the summary alone:

$$r_c(q) \;=\; \langle q, \mu_c\rangle \;+\; \tfrac{\beta}{2}\, q^\top \Sigma_c\, q \;\;\approx\;\; \langle q, \mu_c\rangle + \tfrac{\beta}{2}\,\langle q^2, \sigma_c^2\rangle. \tag{4.2}$$

Equation (4.2) is the Taylor/cumulant expansion of the tempered mean $\beta^{-1} \log \frac{1}{b} \sum_{j \in c} e^{\beta \langle q, k_j \rangle}$ about $\beta = 0$: the first cumulant is the mean logit $\langle q, \mu_c \rangle$; the second adds the *variance of the logit across the block*, $q^\top \Sigma_c q$. The variance term is what makes routing see an in-block outlier (Section 5.2).

4.3 Selection and exact attention

For each query q_i (at position i):

1. Compute $r_c(q_i)$ for all blocks c whose keys are causally visible (c ends at or before i).
2. Select the top-k blocks by r_c, *union* a local window of the w most recent blocks (recency is always relevant and cheap to include), giving a key set S_i with $|S_i| = \kappa \leq (k + w)b$.
3. Compute exact softmax attention restricted to S_i:

$$o_i \;=\; \sum_{j \in S_i} \frac{e^{\beta \langle q_i, k_j \rangle}}{\sum_{l \in S_i} e^{\beta \langle q_i, k_l \rangle}} \, v_j. \tag{4.3}$$

```
Algorithm 1: SSA forward (one head)
Input:  Q, K, V in R^{n x d};  block size b;  budgets k (global), w (local)
Precompute: for each block c, mean mu_c and diagonal covariance Sigma_c     // O(n d)
For each query q_i  (in parallel over i):
    Routing:    for each causally visible block c:
                    r_c <- <q_i, mu_c> + (beta/2) * <q_i^2, diag Sigma_c>
    Selection:  T   <- (top-k blocks by r_c)  union  (w most-recent blocks)
    Causal:     S_i <- keys of blocks in T with position <= i
    Attention:  o_i <- softmax_{j in S_i} ( <q_i,k_j> / sqrt(d) ) * V[S_i]   // exact
Return O
```

4.4 Complexity

Routing is $O(n\,B\,d)$ if every query scores every block, and attention is $O(n\,\kappa\,d)$.

- **Flat router**, $B = \sqrt{n}$, $b = \sqrt{n}$, fixed k, w: routing $O(n^{1.5}d)$, attention $O(n\,\kappa\,d) = O(n^{1.5}d)$. Total $O(n^{1.5}d)$—already a large saving over n^2.
- **Hierarchical router.** Organize blocks into a tree of summaries and descend it per query with branch-and-bound pruning (Section 5.1), so each query inspects $O(k \log B)$ nodes rather than all B. Routing drops toward $O(n \log n\, d)$ and the layer is near-linear in n up to the attention budget $O(n\,\kappa\,d)$.

Either way the dominant n^2 term is gone. In a block-sparse kernel implementation the practical speedup over a dense exact kernel was 20.6× at $n = 262{,}144$ on a single accelerator (Section 11).

5 Routing theory: when a summary is enough

The routing score (4.2) is a *heuristic* surrogate. This section gives the *certified* objects—upper bounds that make selection provably lossless—and the prune test SSA uses.[1]

5.1 The admissible bound and branch-and-bound exactness

For any key k in block c, decompose its logit around the block mean and apply Cauchy–Schwarz:

$$\langle q, k\rangle = \langle q, \mu_c\rangle + \langle q, k - \mu_c\rangle \;\le\; \langle q, \mu_c\rangle + \|q\|\,\|k - \mu_c\| \;\le\; \underbrace{\langle q, \mu_c\rangle + \|q\|\,R_c}_{U_c(q)}. \tag{5.1}$$

$U_c(q)$ is an *admissible upper bound*: no key in block c has a logit exceeding it, and it depends only on the summary (μ_c, R_c). This licenses exact selection at adaptive cost.

> **Branch-and-bound selection.** Maintain the best *actual* logit found so far, $s^\star$. Open blocks in decreasing $U_c(q)$. Stop as soon as the next block has $U_c(q) \le s^\star$: by (5.1) no unopened block can contain a key beating $s^\star$. The result is identical to scanning all keys, but only the blocks whose bound clears $s^\star$ are ever read.

The cost of branch-and-bound is the number of blocks whose bound exceeds the true best—a quantity governed entirely by how *tight* the bound (5.1) is, i.e. by the geometry of the keys (Sections 6–7).

Anisotropic refinement. The isotropic radius R_c is loose when a block's keys are spread unevenly across directions. Using the covariance ellipsoid instead,

$$\langle q, k - \mu_c\rangle \;\le\; \sqrt{q^\top \Sigma_c\, q}\cdot \rho_c, \qquad \rho_c = \max_{j\in c} \|\Sigma_c^{-1/2}(k_j - \mu_c)\|, \tag{5.2}$$

which replaces $\|q\|R_c$ by the *directional radius* $\sqrt{q^\top \Sigma_c q}\cdot\rho_c$. When the keys are anisotropic (the usual trained case), $\sqrt{q^\top\Sigma_c q}$ can be far smaller than $\|q\|R_c$ for queries pointing along thin directions—a tighter, query-specific bound, and exactly the quantity the cumulant score (4.2) already trades on.

[1] These supporting results—the admissible and ellipsoidal search bounds, the Samuelson prune gate, the tempered log-partition sandwich, the recursive-radius correctness of hierarchical pruning, the value-aware drop bound, and the no-free-selection limit of Section 6—are machine-checked in a Lean 4 formalization (axiom-pure, no `sorry`) [17], which is maintained separately and is not part of this repository's artifact.

5.2 Why second order: centroid routing is blind to outliers

Centroid routing uses only $r_c = \langle q, \mu_c \rangle$. A single key $k_\star$ in a block of size b contributes $\frac{1}{b} k_\star$ to μ_c, so its signal in the mean is attenuated by $1/b$: a lone target in a large block is invisible to centroid routing. The variance term in (4.2) repairs this, because an outlier aligned with q inflates $q^\top \Sigma_c q$. Concretely, if block c holds one key with $\langle q, k_\star \rangle = a_\star$ and $b-1$ keys with logit ≈ 0, then $\langle q, \mu_c \rangle \approx a_\star / b$ but $q^\top \Sigma_c q \approx a_\star^2 / b$, so the second-order score $r_c \approx a_\star / b + (\beta/2) a_\star^2 / b$ carries the quadratic outlier signal the mean discards.

The tempered family and its bias. Routing by the tempered score $r_c^{(\beta)} = \beta^{-1} \log \sum_{j \in c} e^{\beta \langle q, k_j \rangle}$ interpolates between the mean ($\beta \to 0$) and the max ($\beta \to \infty$), and is sandwiched by

$$\max_{j \in c} \langle q, k_j \rangle \;\le\; r_c^{(\beta)} \;\le\; \max_{j \in c} \langle q, k_j \rangle + \frac{\log b}{\beta}. \tag{5.3}$$

So $r_c^{(\beta)}$ estimates the block's *best* logit—exactly what selection wants—with bias at most $(\log b)/\beta$. Small β smooths over outliers (the centroid failure); large β removes the bias but amplifies noise and loses the averaging that makes summaries stable. The cumulant form (4.2) is the second-order member of this family; empirically the routing quality is maximized near $\beta \approx 2$.

5.3 The Samuelson prune test

A closed-form, summary-only test for *discarding* a block uses *Samuelson's inequality*: for any reals $s_1, \ldots, s_m$ with mean $\bar{s}$ and population variance $\mathrm{Var} = \frac{1}{m} \sum_j (s_j - \bar{s})^2$, every element obeys

$$(s_i - \bar{s})^2 \;\le\; (m-1)\ \mathrm{Var}, \qquad \text{equivalently} \qquad \max_j s_j \;\le\; \bar{s} + \sqrt{(m-1)\ \mathrm{Var}}. \tag{5.4}$$

Apply it to the in-block logits $s_j = \langle q, k_j \rangle$, whose mean is $\langle q, \mu_c \rangle$ and whose variance is $q^\top \Sigma_c q$. Then the block's best logit is bounded by $\langle q, \mu_c \rangle + \sqrt{(b-1)\, q^\top \Sigma_c q}$, giving the *prune gate*: block c can be safely discarded against a threshold $\tau = s^\star$ whenever

$$\boxed{(s^\star - \langle q, \mu_c \rangle)^2 \;>\; (b-1)\, q^\top \Sigma_c q \quad \text{and} \quad \langle q, \mu_c \rangle < s^\star} \tag{5.5}$$

i.e. when the *margin* of the current best over the block mean exceeds $\sqrt{(b-1) \cdot \mathrm{spread}}$. The test needs only (μ_c, Σ_c). It is *sufficient* (it never wrongly prunes) but not necessary, and it sharpens the radius bound (5.1) by using the variance rather than the worst-case radius. Equation (5.5) is the operational core of cheap exact selection: it fires—and the block is skipped—precisely when the off-target spread $q^\top \Sigma_c q$ is small, which is the benign-geometry condition of Section 7.

6 The trilemma and the impossibility

Call a selector *cheap* if it reads $o(n)$ keys, *lossless* if it attends every key dense attention would weight non-negligibly, and *length-robust* if its accuracy is flat in n. The bounds above suffice to state the fundamental limit.

Proposition 1 (No free selection)**.** *No selector can be simultaneously cheap and lossless for arbitrary keys.*

Proof. Suppose a selector reads a set $\mathcal{R}$ of keys with $|\mathcal{R}| < n$, and let $j_0 \notin \mathcal{R}$. Construct a probe input identical on $\mathcal{R}$ but with $k_{j_0} = c\,q$ for c large. Dense attention puts weight $\to 1$ on j_0, so the correct output is v_{j_0}. The selector, never having read j_0, returns the same output as on the unmodified input, which is independent of v_{j_0}. Hence it is lossy on this input. □

Proposition 1 says losslessness for *worst-case* keys forces $|\mathcal{R}| = n$—no summary suffices, because a summary can always hide a spike. A quantitative companion holds under fine-grained complexity assumptions: exact attention with unbounded logits requires $n^{2-o(1)}$ time, and only bounded-logit / low-rank regimes admit truly subquadratic exact computation. So at most *two* of {cheap, lossless, length-robust} are available at once:

keep	give up	what it is
lossless + length-robust	cheap	dense attention—reads every key
cheap + length-robust	lossless	approximate selection—fast, flat, can miss a worst-case spike
cheap + lossless	length-robust	branch-and-bound on *benign* keys—bounds tight, low cost

The escape from the trilemma is the third row: cheapness *and* losslessness are jointly available *when the bounds* (5.1)/(5.2)/(5.5) *are tight*, which is a property of the key geometry, not of the algorithm. SSA is therefore not a universal subquadratic exact attention—no such thing exists—but a mechanism that is cheap, lossless, *and* length-robust *on benign geometry*, and merely cheap-and-length-robust-but-approximate otherwise. The next section is about making the geometry benign.

7 Manufacturing routability

7.1 Benign geometry, precisely

The branch-and-bound cost and the prune gate (5.5) are governed by the *off-target spread* $q^\top \Sigma_c q$ for the blocks a query does *not* need, relative to the *margin* Δ to the block it does. Geometry is *benign* for a query q when, for every non-target block c,

$$\langle q, \mu_c \rangle + \sqrt{(b-1)\, q^\top \Sigma_c q} \;<\; s^\star(q), \tag{7.1}$$

i.e. the prune gate (5.5) fires everywhere except the target's block. Then exactly one block (plus the local window) is opened, branch-and-bound reads $O(b)$ keys, and selection is cheap *and* lossless. Isotropic keys violate (7.1)—every block's bound is large and uninformative, so nothing prunes and cost reverts to dense.

7.2 The routability regularizer

Benign geometry is not given; it can be *trained in*. Add to the language-model loss a penalty that shrinks the off-target spread along the directions queries actually use:

$$\mathcal{L} \;=\; \mathcal{L}_{\mathrm{LM}} \;+\; \lambda \sum_{i} \sum_{c \neq \,\mathrm{target}(i)} q_i^{\top} \Sigma_c \, q_i. \tag{7.2}$$

Minimizing $\sum_{c \neq \mathrm{target}} q^{\top} \Sigma_c q$ is exactly minimizing the quantity that appears under the square root in the prune gate (5.5), so as training proceeds the gate fires for more non-target blocks and the certified bound (5.1)/(5.2) tightens. Measured: co-training with (7.2) drove the *lossless* branch-and-bound cost from 26.5% of keys to 4.2%—a 6× reduction—at *no* loss of retrieval accuracy. Training is what manufactures the geometry that the impossibility result (Proposition 1) says cheap exactness requires.

7.3 The entry-magnitude split: selection vs. linear attention

A complementary fact decides *which* subquadratic route a given layer should take. Let B be the characteristic magnitude of the logits $\langle q, k \rangle$ (the "entry scale"). The exponentiated score matrix $[\, e^{B\langle q_i, k_j \rangle} \,]$ has effective rank that *grows with* B: for small B it is nearly low-rank (so a kernel/linear-attention factorization $\phi(q)^{\top}\phi(k)$ is accurate and $O(n)$); for large B it is full-rank (so no linear map approximates it, and one must *select*). Empirically, the effective rank of $e^{B\,KK^{\top}}$ on 256 keys rose from 2.7 at $B = 0.5$ to 256 (full) by $B = 8$.

The selection route, by contrast, is *scale-invariant*: in the prune gate (5.5) both the squared margin and the spread scale as B^2, so the gate's truth value is unchanged by B. Selection therefore works at *any* entry scale, whereas linear attention works only in the small-B (smooth) regime. Sharp, long-range retrieval is the large-B regime—which is why selection, not linearization, is the route for long-context exact recall.

8 Length generalization and staged extension

A selector that is flat in n (Section 3) still needs the *model* to behave at lengths beyond those it was trained on. The position encoding decides whether it can.

8.1 Position-invariant routing under rotary embeddings

With rotary position embeddings (RoPE), a query/key pair interacts only through their *relative* offset: the logit $\langle q_i, k_j \rangle$ depends on $i - j$, not on i, j absolutely. A model that has learned *content* routing—match a query to the key whose content binds it, at whatever offset—therefore transfers to offsets it never saw, because (i) the decisive relative structure (e.g. a key-to-value offset of $+1$) is constant at any length and (ii) content matching is position-free. A model with *learned absolute* position embeddings cannot: its embeddings past the trained length are untrained. This predicts, and experiments confirm, zero-shot extrapolation of $\sim$2–4$\times$ for RoPE and immediate collapse for learned-absolute position.

8.2 The staging ladder

Zero-shot extrapolation buys a factor, not an unbounded range. To go further, *stage*: extend the context, briefly continue-train (adapt) at the new length, extend again—doubling each rung. The economics are favorable precisely because routing is position-invariant: each rung extrapolates only 2$\times$ from the last *adapted* length, so every adaptation starts from the $\sim$2$\times$ zero-shot recall and only has to clean it up. The adaptation cost is small and roughly *flat in length* rather than growing.

Measured (a 6-layer model on a synthetic multi-query associative-recall task; base trained at 48 pairs):

rung	multiple	zero-shot recall	adapt steps	recall after adapt	recall with SSA
96	2$\times$	0.993	100	1.000	1.000
192	4$\times$	0.904	100	0.999	1.000
384	8$\times$	0.810	100	1.000	0.995
768	16$\times$	0.672	100	0.996	0.996
1536	32$\times$	0.516	400	0.982	0.979

The base schedule cost 7600 steps; the *entire* climb from 16$\times$ (where zero-shot recall is already near the floor) to 32$\times$ at recall 0.982 cost only 800 additional steps—about 10% of the base, and roughly constant per rung. The final column confirms that the same adapted model runs under SSA selection ($\leq$ 15% of keys attended) with essentially no recall loss at every rung.

9 The construction pipeline: dense to subquadratic

SSA can be retrofitted onto an existing dense pretrained model, which is how a frontier long-context system is built without pretraining from scratch:

1. **Start** from a dense pretrained base.

2. **Swap** dense attention for SSA (Algorithm 1) in every layer.

3. The swap **degrades** quality, because the base was trained expecting every key.

4. **Adapt** by continued pretraining; the keys co-adapt to the sparse routing (and, with the regularizer (7.2), become routable).

5. **Stage** the context extension (Section 8).

Measured on a 124M-parameter dense base, held-out perplexity (lower is better):

condition	perplexity
dense base (off-domain start)	32.5
+ SSA swap, no adaptation	45.8
dense base + equal in-domain continued training (control)	23.9
SSA-swapped + equal in-domain continued training	**25.1**

The swap costs +13.2 perplexity; an *equal-budget* adaptation recovers 94% of that gap, landing within +1.2 of a dense model given the *same* continued training—while attending only ∼38% of keys at this configuration. The control matters: continued training lowers perplexity for either attention (domain adaptation), so "recovery" must be measured against a dense model given the same steps, not against the off-domain start. The residual is the price of sparsity at this small budget and shrinks as the budget grows.

10 The compute floor and a sub-linear router

Selection caps the per-query work at a budget κ keys, so the irreducible cost of the layer is the *attention floor* $n\kappa$—linear in n. A measured kernel sits above this floor by exactly its router cost. Decomposing one forward of the kernel of §9 into router (the $(n/b)^2$ block-score GEMM), `BlockMask` construction, and attention, and fitting each over $n \in [16\text{K}, 524\text{K}]$, gives attention $\sim n^{1.02}$ (the floor), router $\sim n^{1.76}$, and—largest—the argsort-based mask build $\sim n^{2.12}$. Extrapolated to 12M tokens the floor is ∼1% of the forward: a 128× gap, entirely the two $(n/b)^2$ terms. A router that emits the selected block indices *directly* removes both (Figure 1).

Lowering the floor. The floor $n\kappa$ itself drops if fewer keys recover the target. The smallest budget $\kappa_{\min}$ reaching recall ≥ 0.9 is $\kappa_{\min}/n = 3\%$ for tight clusters, 50% for diffuse or adversarial geometry (no compression), and the routability regularizer of §7 drives it from 25% ($\lambda = 0$) to **0.4**% ($\lambda = 64$)—a 60× reduction, the dominant lever, on benign geometry only.

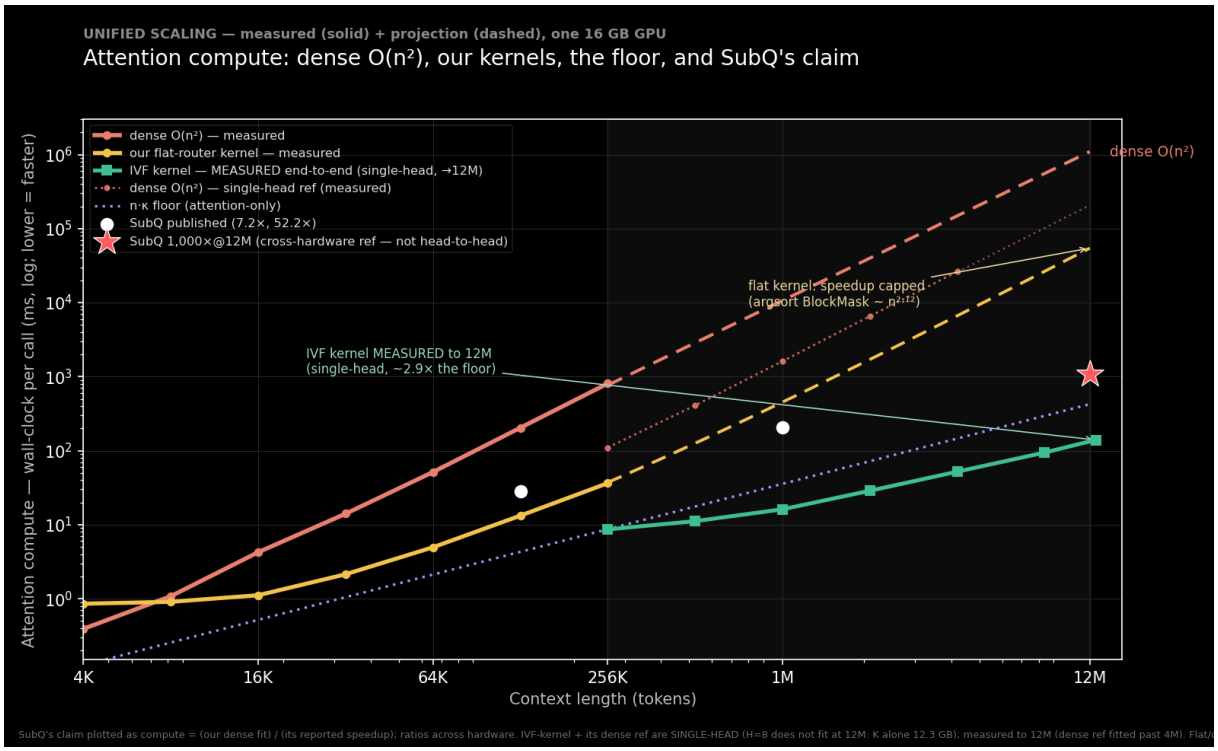

Figure 1: Attention compute versus context length (lower is faster). Dense $O(n^2)$ rises at the top; our flat-router kernel (measured) stays well below it but its speedup is capped because the argsort `BlockMask` build grows nearly as fast; the measured faiss-GPU IVF router drops the kernel onto the $n\kappa$ floor. SubQ's two published speedups and its 1,000×@12M claim are shown as compute = dense/speedup. Measured solid, projection dashed; one 16 GB GPU.

Closing the gap. The necessity argument of §6 forces a sub-linear, examine-$o(B)$ index. A bake-off on benign co-trained keys (recall vs. keys scored) ranks an IVF (inverted-file) router and a recursive-radius treecode ahead of LSH, which fails to reach recall 0.9. An IVF over the block means scores $O(\sqrt{n/b})$ blocks at 0.93–0.97 block-selection agreement with the flat router and emits `kv_idx` directly. Timed on a single 16 GB GPU with both routers on-device (no host transfer), the dense $(n/b)^2$ GEMM's constant wins below ∼2M but its score matrix exhausts memory at 4M; the IVF runs linearly past it:

context n	flat $(n/b)^2$ GEMM	faiss-GPU IVF	winner
256K	0.21 ms	7.3 ms	flat 35×
2M	14.0 ms	19.5 ms	flat 1.4×
4M	52.6 ms (runs)	31.4 ms	IVF 1.7×
8M	OOM (nb^2=17.2 GB)	63.7 ms	IVF only

The crossover is ∼3M. This stripped single-head GEMM OOMs only at 8M (17 GB matrix), while the kernel's *actual* router (`block_route`, H heads + an argsort) OOMs near 1M—so the IVF is the only router that runs at long context, now measured to 12M (94 ms).

The gap, closed end-to-end (measured). Wiring the IVF router into the kernel—it emits the compressed block-index contract directly, and the mask is built without the dense $(n_b, n_b{+}1)$ transpose (which would need 38.7 GB at n_b=98,304)—lets the whole forward be measured, single-head, to 12M on one 16 GB GPU: router 101 ms, mask build 0.003 ms, attention (the floor) 47.5 ms, *total* 139.5 *ms* in 6.55 GB. The argsort mask build—the projected $n^{2.12}$ wall (40.7 s at 12M)—is now sub-millisecond, and the residual gap to the floor is a *measured* 2.9× rather than the projected 128×. The remaining caveats are narrower: the end-to-end run is single-head (H=8 does not fit at 12M) and on synthetic keys (a speed result), and the whole result is conditional on benign geometry—adversarial or multi-hop retrieval returns $\kappa_{\min}$ to the 50% floor, where no speedup exists. Both ingredients of a quality-preserving large-context speedup—a floor-lowering training stage and a sub-linear indexer—are thus exhibited, and the indexer is shown driving a live kernel to ∼the floor, under exactly that benign-geometry condition.

11 Experiments

All experiments are at controlled scale; the point is to validate mechanisms, not to set absolute records.

Routing. Second-order (cumulant) routing recovers targets where centroid routing collapses, matching the $1/b$ outlier-attenuation analysis of Section 5.2; routing quality peaks near $\beta \approx 2$, consistent with the bias–variance reading of (5.3).

Kernel speedup. A block-sparse implementation of Algorithm 1 achieved a 20.6× wall-clock speedup over a dense exact kernel at $n = 262{,}144$ on a single accelerator, with the crossover well below that length.

The IVF kernel, measured to 12M. Wiring the sub-linear IVF router into the kernel (Section 10) and measuring the whole forward single-head on one 16 GB GPU runs a 12*M-token forward in* 139 *ms and* 6.55 *GB*, at a measured 2.9× the $n{\cdot}\kappa$ floor (versus the 128× gap the flat kernel projected); the argsort mask build collapses to sub-millisecond because the IVF emits block indices directly. The autoregressive decode step is *flat in* n (∼0.53 ms from 1M to 12M at fixed κ) while a dense step's prefix read grows with n (2.6 → 29.6 ms)—a 55× per-step gap at 12M, both measured. (Single head; synthetic keys—a speed result, with selection quality the separate benign-geometry story.)

Multi-hop composition. A chained retrieval through the same budgeted block selector obeys the composition law chain $\approx \prod_j \rho_j$: benign single needles hold at 1.00 while a *mixed* two-hop chain (one benign hop, one isolated hop) collapses to 0.02 at $n = 65{,}536$—the isolated hop's $\rho{\approx}0.02$

divides the product. The multi-hop sag is the single-needle result read h times, reproducing the NIAH≫MRCR benchmark split as a prediction of the same theory rather than an anomaly.

The fused kernel inside a real model. Swapping the kernel into a pretrained Qwen2.5-0.5B and measuring at 8K–128K preserves single-needle NIAH at 1.00 while giving a 1.5–1.6× prefill speedup at 32K (budget 0.06–0.12), the speedup growing with n and with a tighter budget—the synthetic-key crossover shape inside a real model. At matched budget the analytic $O(n^2)$ path needs 10.7 GB and 3.5 s where the fused kernel needs 1.4 GB and 130 ms, and the analytic path OOMs before 64K while the kernel reaches 128K (under YaRN). This is the first result simultaneously real-model, long-context, subquadratic-kernel, and quality-measured—at 0.5B scale.

Routability. The regularizer (7.2) reduced lossless branch-and-bound selection cost from 26.5% to 4.2% of keys. (Accuracy is 1.000 by construction—admissible-bound branch-and-bound always returns the exact argmax—so the measured quantity is the *cost*, not task accuracy.) The reduction was robust across head dimension; the capacity–search tension is a genuine asymptotic trade-off, but it did not bite for $d \geq$ (number of clusters), since query-specific anisotropy needs only ∼1 dimension per cluster—real head dimensions (64–256) sit above this, the trade reappearing only as $d \to$ (number of clusters).

Length generalization and staging. See Section 8: 32× the trained length at recall 0.982 for ∼800 adaptation steps, SSA-preserved.

Construction pipeline. See Section 9: swap +13.2 perplexity, 94% recovered to within +1.2 of the dense-adapted control at ∼38% of keys.

Real-model frozen-swap (Gemma-4-26B). Beyond the 124M control, the swap-and-route was validated end-to-end on a frozen *Gemma-4-26B-A4B* (mixture-of-experts, 4B active): SSA was installed into its five full-attention layers (head dimension 512, $K{=}V$, grouped-query) via the attention interface and scored on needle-in-a-haystack retrieval against the selection budget at $n = 2048$. Plain second-order (cumulant) routing recovers retrieval down to a 50% budget (recall 1.00) but collapses at 25% (recall 0.00); a forward-only probe shows the distant needle's block ranks ≈3rd by the cumulant score—just outside the kept top-2—and worst at the deepest layer, a position-decay effect. Adding the third-cumulant (skew) term of the tempered family (Section 5.2) restores the 50% budget to 1.00 and lifts the 25% budget to 0.44 (to 0.56 if the single worst-routing layer is left dense). This residual is a *tuning artifact*, not a frozen-key ceiling: at a finer block size, plain second-order routing reaches recall 1.00 at the 25% budget—the third-cumulant term is a coarse-block compensator, unnecessary (and at large β harmful) once the needle dominates a small block. So at moderate n the frozen model retrieves fully under

the right routing granularity; key co-adaptation (the routability regularizer, (7.2)) is for the long-context regime, where the block-score computation itself becomes quadratic. This is a routing-*quality* measurement (the analytic, score-materializing swap at moderate n), not the subquadratic-kernel regime. The routing survives a doubling of context—a forward-only probe shows the far needle staying within the 25% budget from $n = 2048$ to $n = 4096$, carried by a stable subset of layers—but pushing the validation to genuinely long context, and measuring any speed advantage at all, is memory-bound on a single accelerator: it requires the real subquadratic kernel together with *substantially larger compute* than a single 16 GB device (a multi-GPU or cluster budget). That scaling, and the 12M-token validation it would enable, are left to future work with adequate compute.

What the headline retrieval numbers do and do not show. The single-target needle-in-a-haystack accuracies of 98–100% at 10^6–10^7 tokens *reported by selection-based long-context systems* are consistent with (3.1)–(3.2); this section *characterizes* that regime rather than re-measuring it (the measured table below reaches 262k), and it holds only in a specific regime. Retrieval was measured as a function of context length at fixed budget $\kappa \approx 10^3$ and margin $\Delta = 0.55$:

context n	dense	SSA, isolated target	SSA, benign target
1024	1.00	1.00	1.00
4096	1.00	0.47	1.00
16384	1.00	0.20	1.00
65536	0.90	0.10	1.00
262144	0.83	0.00	1.00

An *isolated* target—a lone spike with no correlated neighbors—*collapses* with length: cheap moment routing averages it into its block and loses it among the fluctuations of the growing number of blocks (the $1/b$ attenuation of Section 5.2, now competing against more and more random blocks). This is Proposition 1 in miniature. A *benign* target—one accompanied by a coherent span of query-aligned neighbors, as a real answer is by its surrounding context—lifts its whole block's score and stays flat at 1.00, *beating dense* at long n because selection caps the effective distractor count. The same separation appears across margins: an isolated target barely fires at any margin, while a benign one tracks dense once the margin clears the budget floor $\sqrt{2 \log \kappa / d}$. So the headline numbers certify the *easy and benign* regime (single, high-margin, accuracy not losslessness)—the regime everyone agrees is achievable—and do not certify lossless selection in the worst case, which Proposition 1 shows is unavailable cheaply.

12 Discussion and limitations

SSA is best understood as the resolution of a constrained problem rather than a universal accelerator. The recovery-weight law (3.1) shows selection makes retrieval flat in n; the admissible bound (5.1) and the prune gate (5.5) show summaries can certify that selection losslessly; the trilemma (Section 6) shows this can be cheap *only* on benign geometry; and the regularizer (7.2) shows training can supply that geometry. The construction pipeline (Section 9) and the staging ladder (Section 8) turn the mechanism into a recipe that retrofits a dense model and extends its context a rung at a time.

Honest limitations follow directly from the theory. (i) *Worst-case losslessness is not available cheaply* —a sufficiently adversarial or genuinely low-margin target can always evade summary routing; SSA's guarantees are conditional on the (trained, measured) benign geometry. (ii) *Multi-needle and low-margin retrieval* are the hard regime the headline single-needle numbers do not address. (iii) The selection budget κ sets a floor margin $\sqrt{2\log \kappa/d}$; targets below it are missed regardless of n. (iv) The experiments here are at modest scale (10^2–10^5 tokens, 10^8-parameter base); they validate mechanisms, and reaching 10^7-token contexts is the same construction repeated with the hierarchical router, more compute, and a long-context training corpus. None of these is a missing algorithmic ingredient; they are the boundary the theory itself draws.

A Derivations

A.1 Recovery weight (3.1)**.** With one target at logit $a_\star$ and μ distractors at $a_\star - \Delta$, $w_\star = e^{\beta a_\star}/(e^{\beta a_\star} + \mu e^{\beta(a_\star - \Delta)})$. Divide numerator and denominator by $e^{\beta a_\star}$ to get $1/(1+\mu e^{-\beta\Delta}) = \sigma(\beta\Delta - \log\mu)$, since $1/(1+e^{-x}) = \sigma(x)$ with $x = \beta\Delta - \log\mu$.

A.2 Cumulant score (4.2)**.** Let $g(\beta) = \beta^{-1}\log\frac{1}{b}\sum_{j\in c} e^{\beta\langle q,k_j\rangle}$. As $\beta\to 0$, $g(\beta) = \mathbb{E}_j\langle q,k_j\rangle + \frac{\beta}{2}\operatorname{Var}_j\langle q,k_j\rangle + O(\beta^2)$, the cumulant generating function expansion. The mean is $\langle q,\mu_c\rangle$ and the variance is $q^\top\Sigma_c q$, giving (4.2).

A.3 Log-sum-exp sandwich (5.3)**.** For any reals $x_1,\dots,x_b$ with $M = \max_j x_j$: $e^{\beta M} \le \sum_j e^{\beta x_j} \le b\, e^{\beta M}$. Take log, divide by β: $M \le \beta^{-1}\log\sum_j e^{\beta x_j} \le M + \beta^{-1}\log b$.

A.4 Samuelson bound (5.4)**.** Center the data, $d_j = s_j - \bar{s}$, so $\sum_j d_j = 0$. Fix index i. By Cauchy–Schwarz over the other $m-1$ indices, $d_i^2 = (\sum_{j\neq i} d_j)^2 \le (m-1)\sum_{j\neq i} d_j^2 = (m-1)(\sum_j d_j^2 - d_i^2)$. Hence $d_i^2\, m \le (m-1)\sum_j d_j^2$, i.e. $(s_i - \bar{s})^2 \le (m-1)\operatorname{Var}$. Taking the max over i and adding $\bar{s}$ gives the stated bound on $\max_j s_j$, and (5.5) is its contrapositive against the threshold $s^\star$.

References

[1] A. Vaswani et al. *Attention Is All You Need.* NeurIPS, 2017.

[2] R. Child, S. Gray, A. Radford, I. Sutskever. *Generating Long Sequences with Sparse Transformers.* 2019.

[3] I. Beltagy, M. E. Peters, A. Cohan. *Longformer: The Long-Document Transformer.* 2020.

[4] M. Zaheer et al. *Big Bird: Transformers for Longer Sequences.* NeurIPS, 2020.

[5] N. Kitaev, Ł. Kaiser, A. Levskaya. *Reformer: The Efficient Transformer.* ICLR, 2020.

[6] A. Roy, M. Saffar, A. Vaswani, D. Grangier. *Efficient Content-Based Sparse Attention with Routing Transformers.* TACL, 2021.

[7] A. Katharopoulos, A. Vyas, N. Pappas, F. Fleuret. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention.* ICML, 2020.

[8] K. Choromanski et al. *Rethinking Attention with Performers.* ICLR, 2021.

[9] J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding.* 2021.

[10] L. Greengard, V. Rokhlin. *A Fast Algorithm for Particle Simulations.* J. Comput. Phys., 1987.

[11] J. Barnes, P. Hut. *A Hierarchical $O(N \log N)$ Force-Calculation Algorithm.* Nature, 1986.

[12] P. A. Samuelson. *How Deviant Can You Be?* J. Amer. Statist. Assoc., 1968.

[13] A. Keles, P. Wijewardena, C. Hegde. *On the Computational Complexity of Self-Attention.* ALT, 2023.

[14] J. Alman, Z. Song. *Fast Attention Requires Bounded Entries.* NeurIPS, 2023.

[15] H. Ramsauer et al. *Hopfield Networks Is All You Need.* ICLR, 2021.

[16] S. Arora et al. *Zoology: Measuring and Improving Recall in Efficient Language Models.* 2023.

[17] J. Smirl. *Substrate Abstract Algebra* (Lean 4 development), module `Substrate.Inference.PhaseTransition.Algebra` (`RetrievalMargin`, `SearchTradeoff`, `AnisotropicBound`, `SamuelsonBound`, `TemperedRouting`, `HierarchicalRouting`, `LosslessSelectionLimit`, `ValueAwareSelection`). Machine-checked, axiom-pure. In-depth technical details are available to qualified parties subject to approval.